IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 31 March 2026, accepted 22 April 2026, date of publication 29 April 2026, date of current version 30 June 2026.

*Digital Object Identifier 10.1109/ACCESS.2026.3688762*

RESEARCH ARTICLE

# Design and Implementation of a Role-Based Hospital Management System Using the MERN Stack with Automated Appointment Booking and Integrated Healthcare Workflow

MUHAMMAD SHOAIB[1], MUHAMMAD ISMAIL MOHMAND[2], NASIR SAYED[1,3], INAYAT KHAN[4], SALMAN JAN[5], AND IT EE LEE[6,7]

[1]Department of Computer Science, CECOS University of IT and Emerging Sciences, Peshawar 25000, Pakistan
[2]Department of Computer Engineering, Faculty of Engineering and Natural Sciences, Istanbul Atlas University, 34408 Istanbul, Türkiye
[3]Department of Computer Science, Islamia College Peshawar, Peshawar 25100, Pakistan
[4]Department of Computer Science, University of Engineering and Technology Mardan, Mardan 23200, Pakistan
[5]Faculty of Computer Studies, Arab Open University, A'ali 732, Bahrain
[6]Faculty of Artificial Intelligence and Engineering, Multimedia University, Cyberjaya 63100, Malaysia
[7]Centre for Smart Systems and Automation, COE for Robotics and Sensing Technologies, Multimedia University, Cyberjaya, Selangor 63100, Malaysia

Corresponding author: It Ee Lee (ielee@mmu.edu.my)

**ABSTRACT** The rapid digital transformation of healthcare has increased the demand for integrated hospital information systems capable of managing clinical and administrative workflows efficiently. Traditional hospital operations often rely on fragmented or manual processes for appointment scheduling, patient record management, prescription handling, and interdepartmental communication, leading to operational inefficiencies and delays in healthcare service delivery. This paper presents **ClinicConnect**, a role-based Hospital Management System designed and implemented using the MERN Stack (MongoDB, Express.js, React.js, and Node.js). The proposed system provides a unified web-based platform that supports five user roles—Patient, Doctor, Receptionist, Pharmacist, and Administrator—through dedicated dashboards with role-based access control. ClinicConnect incorporates automated appointment scheduling, digital prescription management, integrated pharmacy workflow, invoice generation, secure authentication using JSON Web Tokens, cloud-based medical document management, and real-time communication using Socket.IO for notifications and messaging. The system follows a modular component-based architecture that promotes scalability, maintainability, and independent module development while exposing RESTful APIs for seamless client-server communication. The proposed architecture improves coordination among hospital departments by streamlining the complete patient journey from appointment booking to consultation, prescription generation, pharmacy processing, and administrative management. Furthermore, the architecture has been designed to accommodate future enhancements, including AI-assisted prescription processing, intelligent pharmacy document extraction using external AI services, online payment integration, and advanced healthcare analytics. The proposed solution demonstrates how modern web technologies can be utilized to develop a secure, scalable, and extensible digital healthcare platform suitable for hospitals and multi-specialty clinics.

**INDEX TERMS** Hospital Management System, MERN Stack, Healthcare Information System, Appointment Scheduling, Role-Based Access Control, RESTful API, Socket.IO, MongoDB, React.js, Node.js, Express.js.

The associate editor coordinating the review of this manuscript and approving it for publication was Jolanta Mizera-Pietraszko .

## I. INTRODUCTION

The global healthcare sector is in the middle of a sustained digital transformation, driven by rising patient volumes,

tightening resource constraints, and growing expectations that clinical and administrative information be captured, stored, and shared securely. Comprehensive Hospital Information Systems (HIS) are increasingly viewed as necessary infrastructure for coordinating the clinical, financial, and operational data that a modern hospital generates on a daily basis [2]. Despite this, a considerable proportion of small and medium-sized hospitals and multi-specialty clinics continue to rely on manual, paper-based, or only partially digitized processes for patient registration, appointment booking, prescription handling, pharmacy dispensing, billing, and interdepartmental communication, largely because comprehensive enterprise healthcare software remains costly and difficult to customize for smaller facilities [1]. This gap between the promise of digital healthcare and its uneven, fragmented adoption on the ground is the starting point for the work described in this paper.

Fragmentation in hospital operations rarely stays confined to a single department; its effects compound as a patient moves from the front desk to the consultation room to the pharmacy counter. When registration staff record patient details on paper or in a spreadsheet that a doctor cannot later see, and the doctor in turn writes a prescription that the pharmacy has to re-transcribe, each handover becomes an opportunity for transcription errors, lost information, and delay [1]. Reviews of hospital digitalization efforts consistently associate this kind of duplicated data entry and poor interdepartmental communication with longer patient waiting times, inconsistent billing, and slower clinical decision-making [1]. Even where individual departments adopt their own digital tools — a billing spreadsheet here, a scheduling calendar there — systems that were never designed to share data tend to reproduce the same coordination problems in digital form, simply moving the burden of reconciliation from paper to disconnected screens [3]. Addressing this requires more than digitizing individual tasks in isolation; it requires a single platform in which every stakeholder works from the same, continuously updated record of a patient's visit.

Appointment scheduling is one of the clearest examples of this problem. In many hospitals, booking is still handled almost entirely by telephone or in person during limited office hours, a model that restricts patient access, consumes considerable staff time, and is prone to double-booking and manual error [7]. Empirical evaluations of online appointment scheduling report that replacing manual, phone-based booking with automated, availability-aware scheduling measurably reduces the proportion of unused and never-booked appointment slots while improving overall utilization of clinical time [7]. Studies of dedicated hospital appointment platforms similarly show that giving patients real-time visibility into doctor availability, combined with automated confirmations and reminders, shortens effective waiting times and reduces the administrative burden placed on reception staff [5], [8]. Interest in this single workflow has grown to the point that researchers have applied neural-network and machine-learning techniques purely to improve scheduling accuracy, which is itself a signal of how persistent and consequential manual scheduling bottlenecks remain in everyday hospital operations [6].

A parallel and much larger body of research has examined Electronic Health Records (EHR) as the foundation for coordinated, data-driven care. EHR adoption is widely credited with improving the accessibility and continuity of patient information across providers [10]. However, scoping reviews of real-world EHR implementations also document persistent obstacles, including weak governance structures, insufficient interoperability standards, high implementation and maintenance costs, and resistance among staff to changing established work habits [9], [13]. These barriers are not limited to well-resourced health systems: studies of EHR adoption in the Middle East and reviews of health-information-system interoperability across low- and middle-income countries report comparable technical, financial, and organizational constraints, compounded by the challenge of keeping systems affordable and usable outside large hospital networks [11], [12]. For a system intended to serve small and medium-sized facilities, these findings underline that record-keeping cannot be treated as a stand-alone feature; it has to be designed together with authentication, role management, and workflow automation from the outset rather than layered on afterward.

Prescription handling introduces a further, safety-critical dimension to the same problem. Handwritten or verbally transmitted prescriptions are a well-documented source of medication errors, ranging from illegible dosages to missed drug-interaction checks. Clinical evaluations of electronic prescribing systems report substantial reductions in prescribing-error rates after transitioning from paper-based to electronic workflows — one hospital-based study recorded a drop from 1.43% to 0.51% in overall prescribing errors following targeted improvements to its e-prescribing system [14] — and a systematic review and meta-analysis of electronic prescribing strategies across multiple hospitals found a statistically significant reduction in medication errors compared with manual prescribing [15]. Beyond the point of prescribing, many hospitals still rely on a manual handoff between the doctor's note and the pharmacy counter, where dispensing, inventory management, and billing are tracked in separate, uncoordinated systems. An integrated hospital platform therefore needs to treat the prescription not as an isolated document but as a structured record that flows directly from consultation into pharmacy processing, inventory deduction, and invoicing.

These department-level problems have motivated a growing body of work on Hospital Management Systems (HMS) that aim to unify clinical and administrative functions inside one digital platform, a trend sometimes discussed under the broader banner of "Hospital 4.0," in reference to the wider digitalization of medicine [16].

Yet a review of existing HMS literature and implementations shows that most published systems still address only one function at a time — an appointment module here, a records module there, a billing tool elsewhere — rather than an end-to-end platform that connects reception, consultation, pharmacy, and administration under a single, coherent workflow [1]. Several of the appointment-focused systems reviewed above, for example, provide little or no integration with pharmacy or inventory management [5], [8], while EHR-centered systems are rarely designed around the day-to-day operational needs of receptionists or pharmacists [9]. This narrow, single-function focus is the central gap that motivates the present work: hospitals do not need another isolated booking tool or another isolated records module; they need a single, role-aware platform that connects every stakeholder in a patient's journey through the hospital.

The growing maturity of JavaScript-based web technologies has made this kind of unified platform far more practical to build than it would have been a decade ago. The MERN stack — MongoDB, Express.js, React.js, and Node.js — allows a single language to be used across the client, server, and build tooling, while offering a flexible document database, a lightweight RESTful server framework, and a component-based frontend library well suited to building distinct, role-specific dashboards from shared building blocks [4]. The growing number of MERN-based hospital management projects documented in recent literature reflects this suitability: the stack's modular architecture, straightforward horizontal scalability, and large open-source ecosystem make it an attractive choice for institutions that cannot commit to the licensing and infrastructure costs of proprietary enterprise health-IT platforms [4]. Combined with complementary technologies such as JSON Web Token (JWT) authentication, Socket.IO for persistent real-time connections, and cloud object storage for medical documents, the MERN ecosystem provides the pieces needed to build a secure, responsive, and maintainable hospital platform without the overhead of a heavyweight legacy stack.

Because a unified platform concentrates patient, clinical, and financial data behind a single system, access control becomes as important as functionality. Research on access control in health information systems consistently identifies Role-Based Access Control (RBAC) as the baseline mechanism for limiting each user to the data and actions relevant to their responsibilities, whether that user is a physician, a receptionist, an administrator, or a patient [17]. More recent work has extended RBAC with additional layers of encryption and auditability to further protect sensitive medical records from unauthorized access or tampering [18]. These findings reinforce a basic design principle for any hospital platform serving multiple stakeholder types: authentication and authorization cannot be an afterthought bolted onto a working system; they have to be a first-class architectural concern that shapes how every dashboard, API route, and database query is designed from the beginning.

Coordinating five distinct roles around a single patient record also requires more than a conventional request–response API. A receptionist who checks a patient in, a doctor who is finishing a consultation, and a pharmacist who is waiting on a prescription all need to see changes in near real time rather than after a manual page refresh or a phone call between departments. Persistent, event-driven communication technologies such as WebSockets — implemented in the Node.js ecosystem through libraries like Socket.IO — allow a server to push appointment updates, prescription-ready notifications, and administrative broadcasts to every connected dashboard the moment they occur, closing the communication gap that manual and purely request-driven systems leave open between departments.

Taken together, the literature reviewed above points to a consistent and specific gap: hospitals — particularly small and medium-sized institutions without the budget for proprietary enterprise systems — need a single, modular, role-aware platform that carries a patient's information continuously from registration through consultation, prescription, pharmacy processing, and billing, while giving every stakeholder real-time visibility appropriate to their role, all secured behind a consistent authentication and access-control layer. Existing published systems tend to solve one piece of this chain well while leaving the rest to manual coordination or a separate, disconnected tool. Closing this gap is the motivation for the system presented in this paper.

To address this gap, this paper presents ClinicConnect, a role-based Hospital Management System designed and implemented using the MERN Stack (MongoDB, Express.js, React.js, and Node.js) with a modular, component-based architecture. ClinicConnect provides a centralized, web-based platform that digitally connects five stakeholders within a healthcare environment — Patient, Doctor, Receptionist, Pharmacist, and Administrator — each through an independent dashboard tailored to their specific responsibilities and secured through Role-Based Access Control combined with JSON Web Token (JWT) authentication. The system supports the complete patient journey: appointment booking against dynamically generated, transaction-safe time slots that prevent double-booking; doctor consultation and digital prescription generation; an integrated pharmacy workflow covering order processing, inventory, and invoicing; and administrative oversight across departments, doctor schedules, and system-wide activity logs. Real-time notifications, in-app messaging, and hospital-wide broadcasts are handled through Socket.IO, medical reports and other documents are stored securely through cloud-based file management, and outbound communication is handled through an integrated email notification service.

Unlike many of the single-function systems discussed above, ClinicConnect is not limited to a proof-of-concept scheduling widget or an isolated records viewer. A functional backend prototype has been implemented around this architecture, covering role-specific REST APIs and a validated data layer for appointments, doctor availability, prescriptions, pharmacy orders, invoices, medical reports, notifications, and system-wide activity logs, together with the authentication, slot-generation, and real-time communication services needed to operate them. The backend has been structured so that the React-based frontend dashboards for each role can be developed, tested, and scaled independently of one another without requiring changes to the shared API or database layer, an architectural choice intended to keep ClinicConnect easier to extend and maintain than a monolithic hospital application. The architecture has also been deliberately designed to accommodate clearly delineated future enhancements — including AI-assisted prescription support and intelligent pharmacy-document extraction through external AI services, online payment integration, and expanded analytics — as separate modules layered onto the existing system rather than as claims about its current capabilities.
The primary contributions of this work can be summarized as follows:

- Design and implementation of a modular, role-based Hospital Management System using the MERN Stack, structured so that each functional area can be developed, tested, and scaled as an independent unit.
- Development of five dedicated, role-specific dashboards — for Patients, Doctors, Receptionists, Pharmacists, and Administrators — governed by a unified Role-Based Access Control and JWT authentication layer.
- An end-to-end appointment-to-billing workflow, including dynamic, transaction-safe slot generation to prevent double-booking, digital prescription generation, an integrated pharmacy order and inventory workflow, and automated invoice generation.
- A secure RESTful API layer supported by request-level validation and hospital-wide activity logging for auditability, together with a Socket.IO-based real-time communication layer for notifications, broadcasts, and in-app messaging across roles.
- Cloud-based storage for medical reports and other patient documents, integrated alongside automated email notifications for appointment confirmations and administrative alerts.
- A deliberately extensible architecture positioned to accommodate planned future enhancements — including AI-assisted prescription support, intelligent document extraction, online payment integration, and telemedicine — without disrupting the core, currently implemented system.

The remainder of this paper is organized as follows. Section II reviews the existing literature on hospital management systems, digital healthcare platforms, and the specific technologies underlying ClinicConnect. Section III discusses the research problem and motivation in greater detail. Section IV presents the proposed MERN-based system architecture. Section V describes the system design and functional modules for each of the five user roles. Section VI explains the database schema and REST API architecture. Section VII discusses implementation details, including authentication, real-time communication, and cloud storage integration. Section VIII presents the evaluation of the prototype and system-level observations. Finally, Section IX concludes the paper and outlines directions for future work.

## II. LITERATURE REVIEW

The rapid advancement of web technologies has significantly transformed healthcare information systems by enabling hospitals and clinics to digitize clinical and administrative operations. Numerous studies have proposed Hospital Management Systems (HMS), Electronic Health Record (EHR) platforms, and online appointment booking systems to improve healthcare accessibility, patient record management, and hospital efficiency. However, existing research indicates that many healthcare applications continue to focus on individual functionalities such as appointment scheduling, patient record management, or billing rather than providing a unified platform that integrates the complete healthcare workflow.

Several researchers have developed web-based Hospital Management Systems using modern full-stack technologies to replace manual administrative processes. Recent MERN stack implementations demonstrate that integrating MongoDB, Express.js, React.js, and Node.js provides scalable, responsive, and maintainable healthcare applications capable of managing patients, doctors, and administrative operations through a centralized architecture. These systems improve appointment scheduling, reduce paperwork, and provide role-based interfaces for different stakeholders. However, most existing implementations primarily support only Patients, Doctors, and Administrators, while providing limited support for Receptionists, Pharmacists, real-time communication, and integrated hospital workflows.

Online appointment scheduling has become one of the most extensively studied components of digital healthcare platforms. Existing appointment booking systems allow patients to search for doctors, view available schedules, and reserve consultation slots through web interfaces. Automated scheduling significantly reduces patient waiting time, minimizes manual booking errors, and improves doctor availability management compared with conventional paper-based approaches. Recent MERN-based appointment systems further enhance scheduling through RESTful APIs and responsive web interfaces. Nevertheless, many published systems concentrate only on appointment

booking and do not integrate consultation management, prescription generation, pharmacy processing, billing, and administrative monitoring within a single platform.

Electronic patient record management has also received considerable attention in recent years. Digital patient management systems enable healthcare providers to securely maintain patient profiles, medical histories, appointment records, prescriptions, and treatment information within centralized databases. Such systems improve information accessibility, reduce duplicate record creation, and support better clinical decision-making. Despite these advantages, several existing implementations emphasize patient record storage while offering limited interoperability between patient records, appointment scheduling, pharmacy operations, and hospital administration. As a result, healthcare personnel often continue to rely on multiple independent software systems to complete routine clinical workflows.

Modern healthcare applications increasingly adopt three-tier web architectures based on the MERN stack because of their flexibility, scalability, and component-based design principles. React.js enables responsive and interactive user interfaces, Node.js and Express.js provide efficient backend services through RESTful APIs, while MongoDB offers flexible document-oriented storage suitable for healthcare information. Recent research also highlights the importance of modular architectures that separate presentation, business logic, and data management layers to improve maintainability and facilitate future system expansion. Such architectures allow independent development of different hospital modules without affecting the overall system.

Security and access control remain fundamental requirements for healthcare information systems because hospitals manage highly sensitive patient information. Contemporary healthcare applications employ authentication mechanisms such as JSON Web Tokens (JWT), password hashing, and Role-Based Access Control (RBAC) to restrict system access according to user responsibilities. Secure REST APIs, encrypted password storage, and protected routes have become common implementation practices for modern web-based healthcare platforms. Nevertheless, many proposed systems provide only basic authentication while lacking comprehensive role-specific workflow management and real-time coordination among multiple hospital departments.

Real-time communication has emerged as another important research area in digital healthcare. Hospitals increasingly require instant notifications for appointment confirmations, consultation updates, prescription availability, and administrative announcements. Automated email services, notification systems, and event-driven communication significantly improve coordination among patients, doctors, and hospital staff while reducing manual communication efforts. However, most currently available systems either rely exclusively on email notifications or provide only limited real-time synchronization between users and departments. Fully integrated notification frameworks supporting multiple healthcare roles remain relatively uncommon in current hospital management solutions.

Another emerging trend is the integration of Artificial Intelligence into healthcare information systems. Recent studies have investigated AI-assisted medical documentation, intelligent clinical note summarization, and automated prescription processing to reduce physicians' administrative workload and improve healthcare efficiency. Although these technologies demonstrate promising results, their adoption within complete Hospital Management Systems remains limited because of implementation complexity, computational requirements, and integration challenges. Consequently, most current hospital management applications continue to rely primarily on conventional web technologies while treating AI capabilities as optional extensions rather than core system components.

Based on the literature reviewed, it is evident that significant progress has been made in developing digital healthcare platforms. However, several research gaps remain. Most existing systems focus on isolated functionalities such as appointment scheduling, patient record management, or billing without providing a unified, role-based hospital workflow. Many solutions support only a limited number of user roles, lack real-time communication, provide insufficient integration between consultation and pharmacy services, or do not adopt modular software architectures that facilitate future scalability. Furthermore, the integration of intelligent healthcare services, secure cloud-based document management, and event-driven communication remains limited in many existing implementations.

To address these limitations, the proposed ClinicConnect system presents a comprehensive role-based Hospital Management System developed using the MERN Stack. The proposed platform integrates Patients, Doctors, Receptionists, Pharmacists, and Administrators within a unified web-based architecture. It combines automated appointment scheduling, digital prescription management, secure authentication, real-time notifications using Socket.IO, cloud-based medical document storage, billing, and modular RESTful APIs into a single scalable healthcare platform. The architecture has also been designed to accommodate future enhancements such as AI-assisted prescription processing, intelligent pharmacy document extraction, online payment integration, and advanced healthcare analytics without affecting the core functionality of the existing system.

## III. METHODOLOGY

### A. System Workflow

The proposed **ClinicConnect** system follows a modular workflow that digitizes the complete healthcare lifecycle from patient registration to consultation, prescription management, pharmacy services, billing, and administrative supervision. The workflow begins when a user accesses the web application through the React.js frontend and authenticates using secure JWT-based authentication with Role-Based Access Control (RBAC). Based on the authenticated user's role, the system redirects them to their respective dashboard, ensuring that each user accesses only authorized modules and functionalities.

Patients can create an account, update their profile, search for doctors by department or specialization, and view available appointment slots generated according to each doctor's schedule. After selecting a suitable time slot, the appointment request is submitted to the Express.js backend through RESTful APIs. The backend validates the doctor's availability, prevents duplicate bookings, stores the appointment in MongoDB, and generates a confirmation notification. Socket.IO simultaneously delivers real-time appointment updates to the assigned doctor and receptionist, ensuring immediate synchronization across connected users.

Receptionists manage patient registrations, verify appointment requests, coordinate hospital scheduling, and monitor daily appointments through a centralized dashboard. Doctors can review upcoming appointments, access patient medical history, update consultation status, generate digital prescriptions, and recommend medications. Once a prescription is finalized, it becomes instantly available to the pharmacist through real-time notification services.

The complete workflow establishes an integrated digital healthcare environment where every department communicates through a centralized database and secure RESTful APIs, minimizing manual intervention and improving coordination between healthcare professionals.

### B. Methodology of Appointment Scheduling

Appointment scheduling is implemented using a slot-based validation mechanism to prevent scheduling conflicts and duplicate bookings. Every doctor maintains an availability schedule consisting of predefined consultation slots. Whenever a patient requests an appointment, the backend validates the requested slot against existing appointment records stored in MongoDB.

### C. Authentication and Role-Based Access Control

ClinicConnect implements a secure authentication mechanism using JSON Web Tokens (JWT) and bcrypt password hashing. During login, user credentials are verified against encrypted records stored in MongoDB. Upon successful authentication, the backend generates a signed JWT token that is securely transmitted to the client application.

Every incoming API request passes through authentication middleware where the JWT token is verified before granting access to protected resources. The system further implements Role-Based Access Control (RBAC) by assigning permissions according to the authenticated user's role.

Each role has independent dashboard access, dedicated REST APIs, and restricted database operations. Unauthorized requests are rejected before reaching the application logic, thereby enhancing overall system security.

### D. Real-Time Communication Methodology

Real-time communication is implemented using Socket.IO to improve coordination among hospital departments. Unlike traditional polling-based notification systems, Socket.IO establishes persistent bidirectional communication between the client and server.

Whenever significant healthcare events occur, such as appointment creation, appointment confirmation, prescription generation, broadcast messages, or administrative announcements, the backend emits corresponding Socket.IO events to connected users. This mechanism enables instant synchronization of dashboards without requiring manual page refreshes.

The event-driven architecture significantly improves communication efficiency among patients, doctors, receptionists, pharmacists, and administrators while reducing response delays throughout the healthcare workflow.

### F. Overall Methodology

The proposed methodology combines modern web technologies with a modular software architecture to create an integrated healthcare management platform. React.js provides an interactive user interface, Express.js and Node.js implement business logic through RESTful APIs, MongoDB manages healthcare information, and Socket.IO enables real-time communication between hospital departments.

Unlike conventional Hospital Management Systems that automate only isolated administrative activities, ClinicConnect follows an end-to-end healthcare workflow that connects Patients, Doctors, Receptionists, Pharmacists, and Administrators through a unified platform. The modular design improves maintainability, scalability, and future extensibility while supporting additional services such as AI-assisted prescription processing, online payment integration, pharmacy inventory automation, and advanced healthcare analytics

## IV. PROPOSED SYSTEM ARCHITECTURE

### A. Overall Architecture

The proposed **ClinicConnect** system follows a modular three-tier architecture based on the MERN (MongoDB, Express.js, React.js, and Node.js) stack. The architecture is designed to provide scalability, maintainability, and secure communication between different healthcare stakeholders. It consists of three primary layers: the Presentation Layer, the Application Layer, and the Data Layer. Each layer performs independent responsibilities while interacting through RESTful APIs and real-time Socket.IO communication.

The Presentation Layer is developed using **React.js** with **Vite**, providing responsive and interactive user interfaces for Patients, Doctors, Receptionists, Pharmacists, and Administrators. The frontend utilizes reusable components, Context API for state management, Bootstrap 5, and Tailwind CSS to ensure a consistent and responsive user experience across desktops, tablets, and mobile devices.

The Application Layer is implemented using **Node.js** and **Express.js**, which process client requests, implement business logic, authenticate users, validate appointments, generate prescriptions, manage notifications, and expose RESTful APIs. Authentication is secured using JSON Web Tokens (JWT) with Role-Based Access Control (RBAC), ensuring that users can access only the resources associated with their designated roles. Socket.IO is integrated into the backend to support real-time notifications and instant communication between different hospital departments.

The Data Layer utilizes **MongoDB**, a NoSQL document-oriented database that stores patient information, doctor profiles, appointments, prescriptions, medical records, billing information, notifications, chats, inventory, and broadcast messages. MongoDB's flexible schema design enables efficient storage and retrieval of healthcare information while supporting future expansion of additional healthcare services.

The interaction among these layers creates a centralized digital healthcare platform where all hospital operations are synchronized through secure APIs and a shared database, reducing redundancy and improving operational efficiency.

### B. Architecture Components

The proposed architecture consists of the following major components:

**1. Client Layer**

The client layer represents all users interacting with the system through a web browser.

Supported users include:

- Patient
- Doctor
- Receptionist
- Pharmacist
- Administrator

Each user is redirected to an independent dashboard after successful authentication.

**2. Authentication Layer**

The authentication module verifies user credentials using JWT authentication and bcrypt password hashing.

Major responsibilities include:

* User Registration
* Secure Login
* OTP Verification
* Password Encryption
* Token Generation
* Protected Routes
* Role-Based Access Control (RBAC)

This layer ensures that unauthorized users cannot access restricted modules.

**3. Business Logic Layer**

This layer contains the core functionalities of ClinicConnect.

Major services include:

- Appointment Scheduling
- Patient Management
- Doctor Management
- Reception Management
- Pharmacy Management
- Prescription Management
- Billing Management
- Notification Management
- Broadcast Management
- Chat Management

Every request received from the frontend is validated before interacting with the database.

**4. Database Layer**

MongoDB stores all application data using independent collections.

**Major collections include:**

- Users
- Patients
- Doctors
- Receptionists
- Pharmacists
- Admins
- Appointments
- Prescriptions
- Medical Reports
- Medical Records
- Bills
- Notifications
- Chats
- Messages
- Inventory
- Broadcasts

### C. Role-Based Workflow

ClinicConnect follows a role-driven healthcare workflow where each stakeholder performs predefined responsibilities.

**Patient**

- Register/Login
- Search Doctors
- Book Appointment
- View Medical History
- Download Reports
- Receive Notifications

**Receptionist**

- Register Patients
- Manage Appointments
- Verify Bookings
- Coordinate Daily Schedule
- Broadcast Notifications

**Doctor**

- View Daily Appointments
- Access Patient History
- Generate Digital Prescriptions
- Update Consultation Records

**Pharmacist**

- Receive Digital Prescriptions
- Dispense Medicines
- Generate Bills
- Update Inventory
- Complete Medicine Delivery

**Administrator**

- Manage Users
- Manage Departments
- Monitor Hospital Activities
- Configure Doctor Availability
- Generate Reports
- Broadcast Hospital Announcements

### D. Communication Flow

The architecture supports two communication mechanisms:

RESTful API Communication

REST APIs are used for:

- Authentication
- CRUD Operations
- Appointment Booking
- User Management
- Prescription Management
- Billing
- Reports
- Inventory

Approximately **65–80 REST APIs** are planned for the complete implementation.

**Real-Time Communication Socket.IO is responsible for:**

- Appointment Notifications
- New Prescription Alerts
- Broadcast Messages
- Chat System
- Status Updates
- Real-Time Dashboard Synchronization

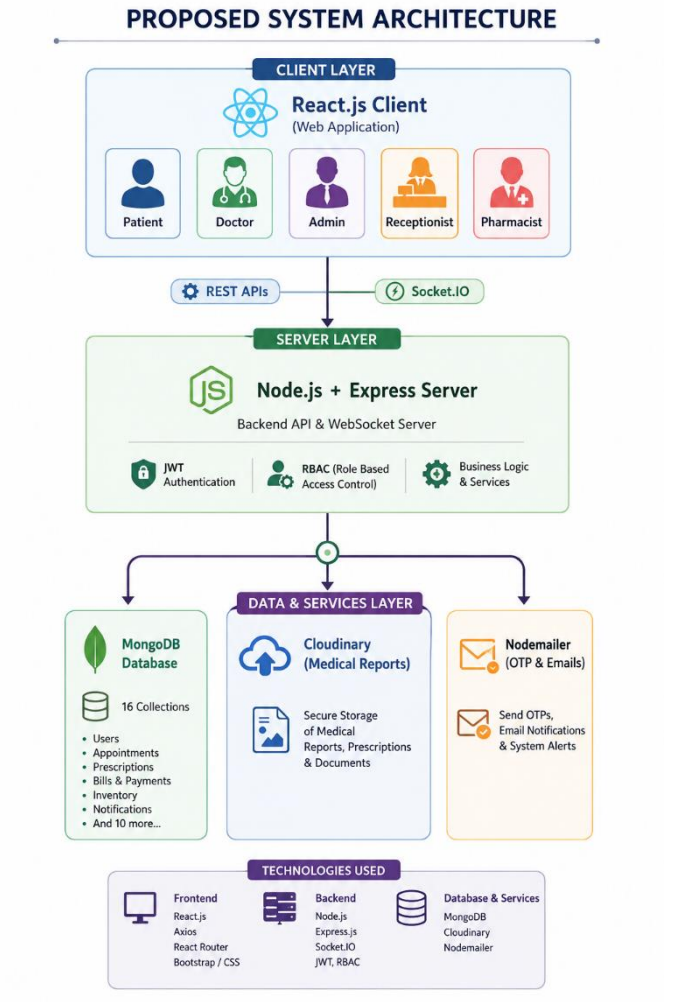
This event-driven communication eliminates the need for continuous client-side polling and improves coordination among users.

### E. Architectural Advantages

The proposed architecture provides several advantages over conventional healthcare management systems.

* Modular and scalable software design
* Independent frontend and backend deployment
* Secure Role-Based Access Control
* Real-time communication using Socket.IO
* Centralized healthcare data management
* Cloud-based medical document storage
* Efficient RESTful API architecture
* Responsive user interface
* Easy maintenance and future scalability
* Integration-ready architecture for AI-assisted healthcare services

The modular separation of presentation, business logic, and data layers enables independent development and testing of each module, simplifying system maintenance and future upgrades.



## V. SYSTEM MODULES

The proposed **ClinicConnect** system is designed using a modular architecture in which each module performs a specific responsibility while communicating with other modules through secure RESTful APIs and a centralized MongoDB database. The modular design improves maintainability, scalability, and independent development of different healthcare services. Every module is protected using Role-Based Access Control (RBAC) to ensure secure access according to user privileges.

### A. Authentication Module

The Authentication Module is responsible for verifying user identity and controlling secure access to the system. It implements JSON Web Token (JWT) authentication, OTP-based verification, bcrypt password hashing, and Role-Based Access Control (RBAC). Users are authenticated before accessing protected resources, and successful authentication redirects them to their respective dashboards based on their assigned roles.

**Major Functionalities**

- User Registration
- Secure Login
- OTP Verification
- JWT Authentication
- Password Encryption using bcrypt
- Forgot Password
- Role-Based Access Control (RBAC)
- Protected Routes

This module ensures that only authenticated users can access healthcare data and prevents unauthorized access to sensitive hospital information.

### B. Patient Management Module

The Patient Management Module provides patients with an interactive dashboard for managing their healthcare activities. Patients can register, maintain personal profiles, search for doctors, book appointments, access medical history, view prescriptions, receive notifications, and communicate with healthcare providers.

**Major Functionalities**

- Patient Registration
- Profile Management
- Search Doctors
- Department-wise Doctor Listing
- Appointment Booking
- Appointment History
- Medical Records
- Digital Prescriptions
- Notifications
- Chat
- View Bills
- Download Medical Reports

This module improves patient convenience by providing digital access to healthcare services from a centralized platform.

### C. Doctor Management Module

The Doctor Module enables doctors to manage consultation schedules and patient interactions efficiently. Doctors can review appointment requests, access patient information, generate digital prescriptions, update consultation records, and communicate with patients and hospital staff.

**Major Functionalities**

- Doctor Dashboard
- Appointment Management
- Availability Management
- Patient Medical History
- Digital Prescription Generation
- Consultation Notes
- Notification Center
- Chat System
- Profile Management

### D. Receptionist Management Module

The Receptionist Module acts as the coordination center of hospital operations. Receptionists manage patient registrations, schedule appointments, verify booking requests, assist patients during hospital visits, and coordinate communication between doctors and patients.

**Major Functionalities**

- Register New Patients
- Appointment Scheduling
- Appointment Verification
- Patient Check-in
- Doctor Schedule Coordination
- Broadcast Notifications
- Billing Coordination
- Dashboard Monitoring

This module minimizes scheduling conflicts and improves patient flow throughout the hospital

### E. Pharmacy Management Module

The Pharmacy Module digitizes prescription processing and medicine distribution. Pharmacists receive prescriptions electronically from doctors, verify medicine availability, generate invoices, dispense medications, and maintain pharmacy records.

**Major Functionalities**

- View Digital Prescriptions
- Medicine Dispensing
- Billing and Invoice Generation
- Inventory Monitoring
- Medicine History
- Patient Prescription Verification
- Notifications

Future versions of this module will integrate AI-assisted prescription understanding and intelligent extraction of medicine information from handwritten prescriptions using external AI services.

### F. Appointment Management Module

Appointment scheduling forms the core functionality of ClinicConnect. The system automatically validates available consultation slots before confirming appointments. Server-side validation prevents duplicate bookings and scheduling conflicts.

**Major Functionalities**

- Book Appointment
- Cancel Appointment
- Reschedule Appointment
- Slot Validation
- Doctor Availability
- Appointment Confirmation
- Appointment Status Tracking
- Automatic Notifications

The appointment management process significantly reduces manual scheduling errors while improving healthcare service efficiency.

### G. Prescription Management Module

The Prescription Module enables doctors to generate digital prescriptions immediately after patient consultation. Prescriptions are securely stored within the centralized database and become instantly available to pharmacists through real-time notifications.

**Major Functionalities**

- Create Digital Prescription
- View Previous Prescriptions
- Medicine Recommendations
- Prescription History
- Prescription Sharing
- Pharmacy Integration

Digital prescription management reduces paperwork while ensuring secure and efficient medicine processing.

### H. Billing and Invoice Module

The Billing Module automates invoice generation for medical consultations and pharmacy services. It records billing information, generates printable invoices, and maintains financial records for administrative purposes.

**Major Functionalities**

- Generate Invoice
- Billing Records
- Patient Billing History
- Payment Status
- Invoice Download

Online payment gateway integration is planned as a future enhancement.

### I. Notification and Communication Module

ClinicConnect implements a real-time notification system using Socket.IO. Notifications are automatically generated whenever appointments are booked, confirmed, cancelled, prescriptions are created, invoices are generated, or broadcast messages are published.

**Major Functionalities**

- Real-Time Notifications
- Appointment Alerts
- Prescription Notifications
- Broadcast Messages
- System Announcements
- Email Notifications

This module ensures timely communication among patients, doctors, receptionists, pharmacists, and administrators.

### J. Chat Module

The Chat Module enables secure communication between authorized users through role-based messaging. It facilitates efficient coordination between hospital staff and improves communication regarding appointments, consultations, and patient management.

**Major Functionalities**

- One-to-One Messaging
- Role-Based Chat
- Real-Time Messaging
- Message History
- Notification Alerts

Socket.IO enables instant synchronization of conversations across connected users.

### K. Administrator Module

The Administrator Module provides complete control over hospital operations. Administrators manage users, departments, doctors, appointments, broadcasts, notifications, reports, and overall system configuration.

**Major Functionalities**

- User Management
- Department Management
- Doctor Management
- Appointment Monitoring
- Broadcast Management
- Notification Management
- Report Monitoring
- Dashboard Analytics
- System Configuration

This centralized control enables efficient supervision of all healthcare operations.

### L. Future Enhancement Module

The modular architecture of ClinicConnect supports future integration of advanced healthcare technologies without requiring major structural modifications.

**Future enhancements include:**

- AI-assisted prescription understanding using external AI APIs.
- Intelligent extraction of medicine information from handwritten prescriptions.
- Online payment gateway integration.
- Advanced pharmacy inventory management.
- Healthcare analytics and reporting dashboards.
- Telemedicine and video consultation support.
- Mobile application support for Android and iOS platforms.

## VI. DATABASE DESIGN

### A. Database Overview

The proposed **ClinicConnect** system utilizes **MongoDB**, a NoSQL document-oriented database, as the primary data storage solution. MongoDB was selected because of its flexible schema design, scalability, high performance, and seamless integration with the MERN technology stack. Unlike traditional relational databases, MongoDB stores data in JSON-like BSON documents, allowing healthcare information with varying structures to be managed efficiently.

The database is designed to support multiple user roles,

appointment scheduling, prescription management, pharmacy operations, billing, notifications, chat services, and administrative functions. Collections are logically separated according to functional modules, improving maintainability and reducing data redundancy. Relationships between collections are maintained using **MongoDB ObjectId references** and Mongoose population mechanisms to retrieve related information efficiently.

The proposed database architecture consists of **16 collections**, each representing a major entity within the healthcare workflow.

**B. Database Collections**

**1. Users**

Stores authentication credentials and common user information.

**Attributes**

- User ID
- Full Name
- Email
- Mobile Number
- Password (bcrypt hashed)
- Role
- Account Status
- Created At

**2. Patients**

Maintains patient-specific information.

**Attributes**

- Patient ID
- User ID
- Date of Birth
- Gender
- Blood Group
- Address
- Emergency Contact
- Medical History

**3. Doctors**

Stores professional information related to doctors.

**Attributes**

- Doctor ID
- User ID
- Specialization
- Qualification
- Experience
- Consultation Fee
- Availability Schedule

**4. Receptionists**

Maintains receptionist information and assigned responsibilities.

**Attributes**

- Receptionist ID
- User ID
- Department
- Contact Information

**5. Pharmacists**

Stores pharmacist details.

**Attributes**

- Pharmacist ID
- User ID
- Qualification
- Experience
- Assigned Pharmacy

**6. Administrators**

Stores administrator profile information.

**Attributes**

- Admin ID
- User ID
- Designation
- Access Permissions

**7. Appointments**

Stores all appointment-related information.

**Attributes**

- Appointment ID
- Patient ID
- Doctor ID
- Appointment Date
- Time Slot
- Status
- Reason for Visit
- Created At

**8. Prescriptions**

Stores digital prescriptions generated after consultation.

**Attributes**

- Prescription ID
- Appointment ID
- Patient ID
- Doctor ID
- Medicine List
- Dosage
- Instructions
- Date Issued

**9. Medical Reports**

Stores uploaded medical reports.

**Attributes**

- Report ID
- Patient ID
- File URL
- Report Type
- Upload Date

Cloudinary is used to store report files securely.

**10. Medical Records**

Maintains long-term healthcare history.

**Attributes**

- Record ID
- Patient ID
- Diagnosis
- Treatment
- Allergies
- Previous Illnesses

**11. Bills**

Stores invoice and billing information.

**Attributes**

- Bill ID
- Patient ID
- Appointment ID
- Total Amount

- Payment Status
- Invoice Date

**12. Notifications**

Stores notification history.

**Attributes**

- Notification ID
- Sender ID
- Receiver ID
- Message
- Notification Type
- Status
- Timestamp

**13. Chats**

Maintains chat sessions between users.

**Attributes**

- Chat ID
- Participants
- Last Message
- Created At

**14. Messages**

Stores chat messages.

**Attributes**

- Message ID
- Chat ID
- Sender ID
- Receiver ID
- Message Content
- Timestamp

**15. Inventory**

Stores pharmacy medicine inventory.

**Attributes**

- Medicine ID
- Medicine Name
- Category
- Quantity
- Expiry Date
- Supplier
- Price

**Note:** Advanced inventory prediction and AI-assisted inventory analysis are planned as future enhancements.

**16. Broadcasts**

Stores hospital-wide announcements.

**Attributes**

- Broadcast ID
- Sender
- Target Role
- Title
- Message
- Created At

### C. Collection Relationships

The proposed database maintains logical relationships among collections using MongoDB ObjectId references.

The major relationships are summarized below:

- One **User** corresponds to one Patient, Doctor, Receptionist, Pharmacist, or Administrator profile.
- One **Patient** can book multiple Appointments.
- One **Doctor** can attend multiple Appointments.
- One Appointment generates one or more Prescriptions.
- One Patient can have multiple Medical Reports and Medical Records.
- One Patient can generate multiple Bills.
- Notifications are associated with individual users.
- Chat sessions contain multiple Messages.
- Pharmacists process Prescriptions and update Inventory records.

### D. Database Advantages

The proposed MongoDB database architecture provides several advantages for healthcare information management.

- Flexible schema design for evolving healthcare data.
- High scalability for increasing patient records.
- Efficient integration with the MERN Stack.
- Reduced data redundancy through modular collections.
- Fast retrieval using indexed ObjectId references.
- Secure storage of sensitive healthcare information.
- Easy expansion for future healthcare modules.
- Cloud-based document integration through Cloudinary.
- Support for real-time updates using Socket.IO.

## VII. IMPLEMENTATION

### A. Implementation Overview

The proposed **ClinicConnect** system is implemented as a full-stack web application using the **MERN Stack**, consisting of MongoDB, Express.js, React.js, and Node.js. The system follows a modular architecture in which the frontend, backend, and database operate independently while communicating through secure RESTful APIs. This separation improves maintainability, scalability, and simplifies future feature integration.

The frontend is developed using **React.js** with **Vite**, providing a responsive and interactive user interface for Patients, Doctors, Receptionists, Pharmacists, and Administrators. The backend is implemented using **Node.js** and **Express.js**, which manage authentication, business logic, API routing, appointment scheduling, prescription processing, notifications, and communication between different system modules. MongoDB serves as the primary database for storing healthcare information, while **Mongoose ODM** is used for schema modeling and database operations.

The application follows a client-server architecture where the frontend communicates with the backend through RESTful APIs, and the backend interacts with MongoDB for persistent data storage. Real-time communication is achieved using **Socket.IO**, enabling instant notifications and synchronized updates across connected users.

### B. Technology Stack

The implementation utilizes modern web technologies to ensure high performance, security, and scalability.

| Component | Technology Used |
|---|---|
| Frontend | React.js + Vite |
| Backend | Node.js + Express.js |
| Database | MongoDB |
| ODM | Mongoose |
| Authentication | JWT + bcrypt |
| State Management | Context API |
| Styling | Bootstrap 5 + Tailwind CSS |
| Real-Time Communication | Socket.IO |
| File Storage | Cloudinary |
| Email Service | Nodemailer |
| API Architecture | RESTful APIs |

This technology stack enables independent development of frontend and backend components while supporting efficient communication through standardized HTTP requests.

### C. Frontend Implementation

The frontend of ClinicConnect is implemented using **React.js**, adopting a component-based architecture to improve code reusability and maintainability. User interfaces are organized into reusable components, pages, layouts, and role-specific dashboards.

React Router is utilized for client-side navigation between different modules, while Context API manages application-wide state such as user authentication, profile information, and notification status. Responsive user interfaces are developed using Bootstrap 5 and Tailwind CSS, ensuring compatibility across desktops, tablets, and mobile devices.

Each authenticated user is redirected to a dedicated dashboard based on their assigned role, enabling secure access to module-specific functionalities.

### D. Backend Implementation

The backend is developed using **Node.js** and **Express.js**, following a modular MVC architecture. Business logic is organized into controllers, services, middleware, and route handlers to maintain clear separation of responsibilities.

The backend exposes approximately **65–80 RESTful APIs** covering authentication, patient management, doctor management, receptionist operations, pharmacy services, appointment scheduling, prescriptions, billing, notifications, and administrative functions.

Incoming requests are validated through middleware before interacting with the database. Error handling mechanisms ensure consistent API responses and improve system reliability.

### E. Authentication and Authorization

ClinicConnect implements secure authentication using **JSON Web Tokens (JWT)**. During user login, credentials are validated against encrypted passwords stored in MongoDB. Passwords are hashed using **bcrypt**, preventing storage of plaintext credentials.

After successful authentication, the server generates a JWT access token that is included in subsequent API requests. Protected middleware verifies the authenticity of the token before allowing access to secured resources.

Role-Based Access Control (RBAC) is implemented to ensure that each user accesses only the functionalities associated with their designated role.

The supported roles include:

- Patient
- Doctor
- Receptionist
- Pharmacist
- Administrator

Unauthorized API requests are rejected with appropriate HTTP status codes.

### F. Appointment Scheduling Implementation

Appointment scheduling is one of the core modules of ClinicConnect. Patients can search for doctors based on specialization, select available consultation slots, and submit appointment requests through the web interface.

The backend validates doctor availability before confirming the booking. If the requested slot is already reserved, the system rejects the request and prompts the patient to choose another available slot.

Upon successful booking:

- Appointment details are stored in MongoDB.
- Socket.IO generates real-time notifications.
- Doctors and Receptionists receive instant updates.
- Patients receive appointment confirmation.

This mechanism minimizes scheduling conflicts and ensures efficient appointment management.

### G. Prescription and Pharmacy Implementation

Doctors generate digital prescriptions after completing patient consultations. Prescription information is securely

stored within the MongoDB database and immediately becomes accessible to authorized pharmacists.

Pharmacists can:

- View prescriptions.
- Verify medication details.
- Prepare medicine invoices.
- Update dispensing status.

The modular design enables future integration of AI-assisted prescription understanding using external AI APIs without modifying the existing prescription workflow.

**H. Notification and Communication Implementation**

ClinicConnect integrates **Socket.IO** to provide event-driven real-time communication between different stakeholders.

Notifications are automatically generated for events such as:

- Appointment Booking
- Appointment Confirmation
- Appointment Cancellation
- Prescription Generation
- Broadcast Messages
- Administrative Announcements

Socket.IO establishes persistent bidirectional communication between the server and connected clients, ensuring that dashboard information remains synchronized without requiring manual page refreshes.

**I. Medical Report Management**

Medical documents are stored using **Cloudinary**, providing secure cloud-based storage for healthcare records.

Patients and doctors can upload, view, and download medical reports through authenticated interfaces. Cloudinary stores only file references within MongoDB, reducing database storage overhead while improving file accessibility and security.

**J. Billing and Invoice Generation**

The billing module automatically generates invoices based on completed consultations and prescribed medications.

Billing information includes:

- Consultation Charges
- Medicine Charges
- Invoice Number
- Billing Date
- Payment Status

Invoice records are stored within MongoDB and can be retrieved through the respective patient and pharmacist dashboards.

Future versions of the system will integrate secure online payment gateways for digital payment processing.

**K. Real-Time Chat System**

ClinicConnect includes a role-based chat module implemented using Socket.IO.

Authorized users can communicate through secure messaging channels.

Supported communication includes:

- Patient ↔ Doctor
- Doctor ↔ Receptionist
- Receptionist ↔ Administrator
- Pharmacist ↔ Administrator

Messages are stored within MongoDB, enabling users to access previous conversations whenever required.

**L. System Security**

Security has been incorporated throughout the implementation.

Major security mechanisms include:

- JWT Authentication
- Role-Based Access Control (RBAC)
- Password Hashing using bcrypt
- Protected REST APIs
- Secure File Storage
- Input Validation
- Error Handling
- Environment Variable Configuration

These mechanisms collectively protect sensitive healthcare information and reduce the risk of unauthorized system access.

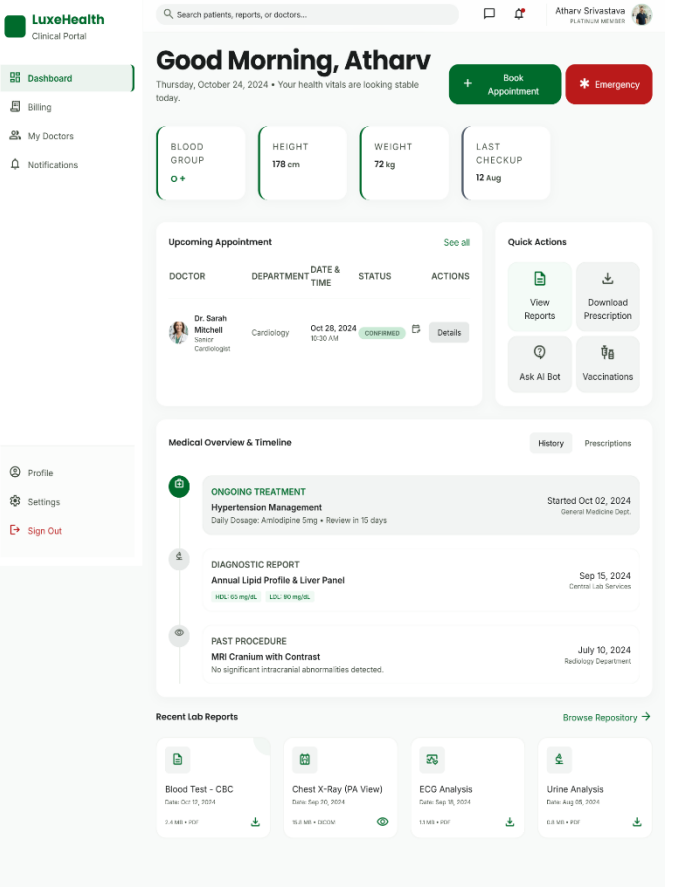
**M. Implementation Summary**

The implementation of ClinicConnect demonstrates how modern web technologies can be integrated to develop a secure, scalable, and modular Hospital Management System. The combination of React.js, Node.js, Express.js, MongoDB, Socket.IO, JWT authentication, and Cloudinary enables efficient management of healthcare workflows while supporting future expansion through additional healthcare services.

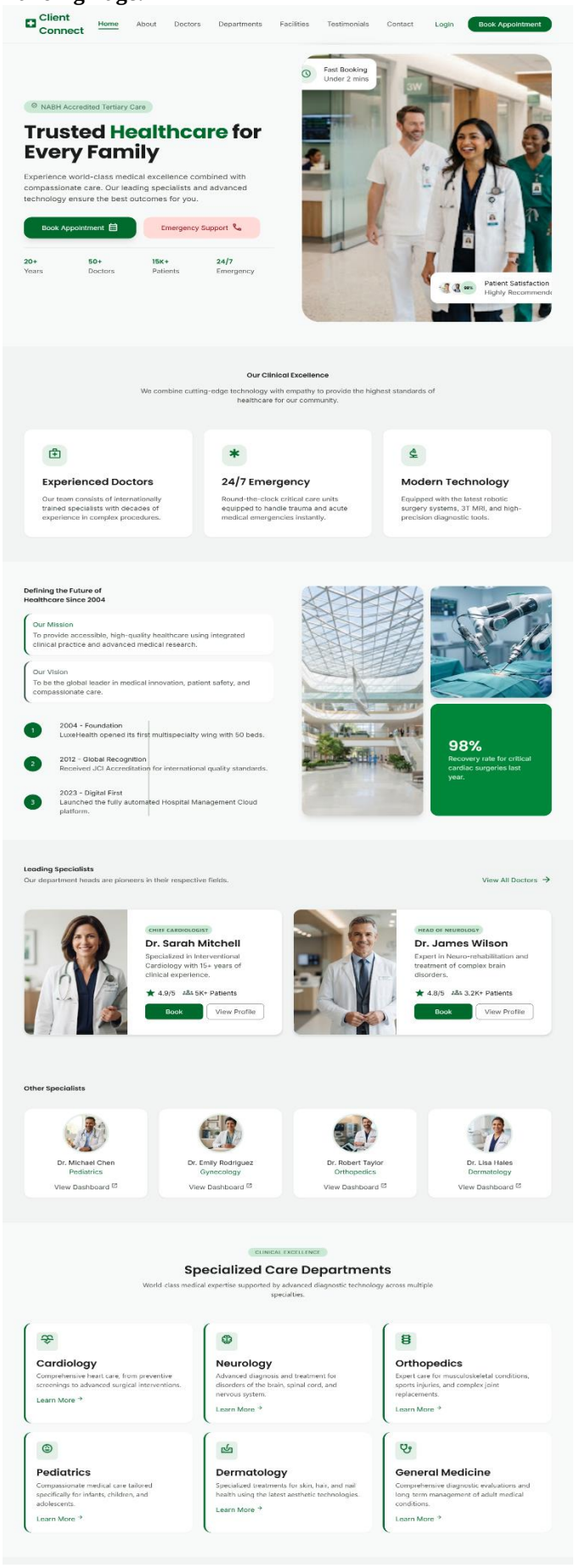
The modular architecture facilitates independent development of different system components and ensures that new functionalities can be incorporated with minimal impact on the existing application.

**N. Implementation Screenshot**

**Patient Dashboard:-**



**Landing Page:-**




## VIII. EXPERIMENTAL RESULTS AND EVALUATION

### A. Experimental Setup

The proposed **ClinicConnect** system was developed and evaluated in a local development environment using the MERN Stack. Functional testing was conducted to verify the correctness of individual modules and the interaction between different healthcare roles. The application was executed using React.js with Vite as the frontend framework, Node.js and Express.js as the backend runtime, and MongoDB as the database server.

The evaluation focused on validating the core functionalities of the proposed system rather than benchmarking hardware performance. Each functional module was tested independently and then integrated into the complete healthcare workflow to ensure smooth communication between the frontend, backend, and database.

### B. Functional Validation

Functional testing was performed on all major modules to verify that the implemented functionalities behaved according to the proposed system requirements.

**Table 5. Functional Testing Results**

| Module | Function Tested | Status |
|---|---|---|
| Authentication | User Login & Registration | ✓ Passed |
| Authentication | JWT Verification | ✓ Passed |
| Patient Module | Profile Management | ✓ Passed |
| Patient Module | Appointment Booking | ✓ Passed |
| Doctor Module | Appointment Management | ✓ Passed |
| Doctor Module | Digital Prescription | ✓ Passed |
| Reception Module | Appointment Coordination | ✓ Passed |
| Pharmacy Module | Prescription Verification | ✓ Passed |
| Billing Module | Invoice Generation | ✓ Passed |
| Notification Module | Real-Time Notification | ✓ Passed |
| Chat Module | Real-Time Messaging | ✓ Passed |
| Admin Module | User Management | ✓ Passed |

The successful execution of these modules confirms that the proposed architecture effectively supports the complete healthcare workflow from patient registration to consultation, prescription processing, and administrative management.

### C. Workflow Validation

The proposed healthcare workflow was evaluated by simulating interactions among different hospital stakeholders.

The validated workflow follows the sequence below:

1. Patient Registration
2. Secure Login
3. Doctor Search
4. Appointment Booking
5. Appointment Confirmation
6. Doctor Consultation
7. Digital Prescription Generation
8. Pharmacy Processing
9. Invoice Generation
10. Notification Delivery

The workflow was successfully completed without requiring manual synchronization between departments, demonstrating the effectiveness of the integrated architecture.

### D. Security Evaluation

The security mechanisms implemented in ClinicConnect were verified through functional testing.

The following security features were successfully validated:

- JWT Authentication
- Role-Based Access Control (RBAC)
- Password Hashing using bcrypt
- Protected API Routes
- Unauthorized Access Prevention
- Secure Session Management

Unauthorized requests without valid authentication tokens were denied access to protected resources, ensuring that only authenticated users could perform authorized operations.

### E. Real-Time Communication Evaluation

Socket.IO was evaluated by monitoring real-time communication between different user roles.

The following events were successfully synchronized:

- Appointment Booking Notifications
- Appointment Confirmation
- Digital Prescription Availability
- Broadcast Messages
- Chat Messages
- Administrative Notifications

The event-driven communication architecture eliminated the need for repeated client-side polling and ensured that connected users received updates immediately after server-side events occurred.

### F. Database Evaluation

MongoDB successfully managed healthcare information across multiple collections.

The implemented database architecture supported:

- User Authentication
- Appointment Records
- Prescription Storage
- Medical Records
- Billing Information
- Notifications
- Chat Messages
- Broadcast Messages

### G. Discussion

The implementation demonstrates that a modular MERN-based architecture can effectively support multiple healthcare stakeholders through a centralized digital platform. By integrating appointment scheduling, consultation management, digital prescriptions, billing, notifications, and role-based dashboards, ClinicConnect minimizes manual administrative effort while improving coordination among hospital departments.

The use of React.js, Express.js, MongoDB, and Socket.IO enables efficient communication between system modules while maintaining flexibility for future enhancements. The modular architecture also allows additional healthcare services, such as AI-assisted prescription understanding, pharmacy inventory automation, telemedicine, and online payment integration, to be incorporated without significant architectural changes.

Although the proposed system successfully achieves the intended objectives, certain advanced features, including AI-assisted prescription processing, intelligent pharmacy document extraction, advanced inventory analytics, and online payment integration, remain planned for future implementation.

### H. Overall Evaluation

The experimental evaluation confirms that ClinicConnect successfully fulfills the primary objectives of developing a secure, scalable, and role-based Hospital Management System using the MERN Stack.

The implemented system provides:

- Secure Authentication
- Role-Based Access Control
- Automated Appointment Scheduling
- Digital Prescription Management
- Integrated Billing
- Real-Time Notifications
- Secure Cloud Storage
- Modular RESTful Architecture

The results indicate that the proposed architecture provides an effective foundation for digital healthcare management while supporting future technological advancements.

## IX. CONCLUSION AND FUTURE WORK

### A. Conclusion

This paper presented **ClinicConnect**, a role-based Hospital Management System designed and implemented using the MERN Stack to digitalize and streamline healthcare operations within hospitals and multi-specialty clinics. The proposed system integrates multiple healthcare services, including patient management, doctor consultation, appointment scheduling, digital prescription management, pharmacy workflow, billing, notifications, and administrative control into a unified web-based platform. By adopting a modular architecture and Role-Based Access Control (RBAC), the system provides dedicated dashboards for Patients, Doctors, Receptionists, Pharmacists, and Administrators while ensuring secure and efficient access to healthcare information.

The implementation demonstrates the effectiveness of modern web technologies such as React.js, Node.js,

Express.js, MongoDB, JWT authentication, Socket.IO, Cloudinary, and RESTful APIs in developing a scalable and maintainable healthcare management solution. The modular design enables seamless communication between different hospital departments, reduces manual administrative tasks, minimizes appointment scheduling conflicts, and improves the accessibility of healthcare information through centralized data management.

The functional evaluation confirms that the proposed system successfully supports secure authentication, appointment scheduling, prescription generation, billing, real-time notifications, and role-based workflow management. The separation of presentation, application, and database layers enhances system maintainability while allowing independent development and future expansion of individual modules. The proposed architecture provides a practical foundation for healthcare institutions seeking an integrated digital platform capable of improving operational efficiency and patient service delivery.

### B. Future Work

Although the current implementation provides a comprehensive digital healthcare platform, several advanced features can be incorporated in future versions to further enhance the system's capabilities.

Future enhancements include:

- Integration of **AI-assisted prescription understanding** using external AI APIs to extract medicine information from handwritten or uploaded prescriptions.
- Automated **prescription document analysis** to assist pharmacists in verifying medicine details and improving prescription processing efficiency.
- **Online payment gateway integration** for secure digital payment of consultation fees and medical invoices.
- **Advanced pharmacy inventory management** with automated stock monitoring and low-stock alerts.
- **Healthcare analytics dashboards** for monitoring hospital performance, patient statistics, appointment trends, and resource utilization.
- **Telemedicine and video consultation** support to enable remote healthcare services.
- **Mobile application development** for Android and iOS platforms to improve accessibility for patients and healthcare professionals. .
- Support for **multi-hospital and multi-branch deployment**, enabling centralized administration across multiple healthcare facilities.

## REFERENCES

[1] D. J. Eden, A. Hermann, L. B. Sombrotto, P. J. Wilner, and J. A. Chen, ''Finding our lanes: A roadmap for collaboration between academic medical centers and behavioral telehealth companies,'' *Harvard Rev. Psychiatry*, vol. 32, no. 4, pp. 140–149, 2024

[2] R. Singh, T. G. Guan, K. Ravesangar, and A. Joshi, ''Artificial intelli-gence adoption for sustainable HRM in HealthCare industry: A conceptual review,'' *Transform. Healthc. Sect. Through Artif. Intell. Environ. Sustain.*, vol. 3, pp. 231–251, Jan. 2025.

[3] G. Nwaogbe, E. Ekpenyong, and O. Urhoghide, ''Managing workforce productivity in the post-pandemic construction industry,'' *World J. Adv. Res. Rev.*, vol. 25, no. 1, pp. 572–588, Jan. 2025.

[4] S. A. Karim and M. J. Islam, ''Navigating the future of health-care HR: Agile strategies for overcoming modern challenges,'' 2024, *arXiv:2410.04246*.

[5] V. Kulkarni, S. Darak, R. Parchure, A. Paranjape, and S. Barat, ''Building resilient public healthcare systems using digital twins,'' in *Digital Twins for Simulation-Based Decision-Making*. Cham, Switzerland: Springer, 2025, pp. 195–222.

[6] S. Sharmin, ''Refugee resettlement & AI-powered resource allocation optimizing social services for displaced populations,'' *J. Public Adm. Res.*, vol. 2, no. 1, pp. 13–36, 2025.

[7] P. Świtaj, Ł. Baka, Ł. Kapica, J. Bugajska, J. Januszczak, and A. Sienkiewicz-Jarosz, ''Insomnia mediates the longitudinal association between loneliness and burnout among nurses,'' *Sci. Rep.*, vol. 15, no. 1, p. 25384, Jul. 2025.

[8] F. Li, S. Wang, Z. Gao, M. Qing, S. Pan, Y. Liu, and C. Hu, ''Har-nessing artificial intelligence in sepsis care: Advances in early detection, personalized treatment, and real-time monitoring,'' *Frontiers Med.*, vol. 11, Jan. 2025, Art. no. 1510792.

[9] A. Pandey, M. Maheshwari, and N. Malik, ''A systematic literature review on employee well-being: Mapping multi-level antecedents, moderators, mediators and future research agenda,'' *Acta Psychologica*, vol. 258, Aug. 2025, Art. no. 105080.

[10] A. M. Antolí-Jover, M. A. Álvarez-Serrano, M. Gázquez-López, A. Martín-Salvador, M. Á. Pérez-Morente, E. Martínez-García, and I. García-García, ''Impact of work-life balance on the quality of life of Spanish nurses during the sixth wave of the COVID-19 pan-demic: A cross-sectional study,'' *Healthcare*, vol. 12, no. 5, p. 598, Mar. 2024.

[11] M. Kroczek and J. Späth, ''The attractiveness of jobs in the German care sector: Results of a factorial survey,'' *Eur. J. Health Econ.*, vol. 23, no. 9, pp. 1547–1562, Dec. 2022.

[12] B. K. Sharma, ''AI managed hospital workforce,'' in *Role of Artificial Intelligence, Telehealth, and Telemedicine in Medical Virology*. Cham, Switzerland: Springer, 2025, pp. 221–251.

[13] P. Gupta, D. Gupta, and M. Gupta, ''Unleashing human capital analytics for data-driven workforce management,'' *Hum. Cap. Anal. Explor. HR Spectr. Ind. 5*, vol. 9, pp. 143–163, Jun. 2025.

[14] M. Yazdani, S. Shahriari, and M. Haghani, ''Real-time decision support model for logistics of emergency patient transfers from hospitals via an integrated optimisation and machine learning approach,'' *Prog. Disaster Sci.*, vol. 25, Jan. 2025, Art. no. 100397.

[15] K. Ramar, A. S. Oxentenko, and S. C. Dowdy, ''Transforming health care through quality and safety,'' *Mayo Clinic Proc.*, vol. 100, no. 8, pp. 1385–1401, Aug. 2025.

[16] M. Faiyazuddin, S. J. Q. Rahman, G. Anand, R. K. Siddiqui, R. Mehta, M. N. Khatib, S. Gaidhane, Q. S. Zahiruddin, A. Hussain, and R. Sah, ''The impact of artificial intelligence on healthcare: A comprehensive review of advancements in diagnostics, treatment, and operational efficiency,'' *Health Sci. Rep.*, vol. 8, no. 1, Jan. 2025, Art. no. e70312.

[17] M. Alves, J. Seringa, T. Silvestre, and T. Magalhães, ''Use of arti-ficial intelligence tools in supporting decision-making in hospital management,'' *BMC Health Services Res.*, vol. 24, no. 1, p. 1282, Oct. 2024.

[18] B.-J. Kim and M.-J. Kim, ''How artificial intelligence-induced job inse-curity shapes knowledge dynamics: The mitigating role of artificial intelligence self-efficacy,'' *J. Innov. Knowl.*, vol. 9, no. 4, Oct. 2024, Art. no. 100590.

[19] M. F. Shahzad, S. Xu, W. Naveed, S. Nusrat, and I. Zahid, ''Investigating the impact of artificial intelligence on human resource functions in the health sector of China: A mediated moderation model,'' *Heliyon*, vol. 9, no. 11, Nov. 2023, Art. no. e21818.

[20] V. Prikshat, M. Islam, P. Patel, A. Malik, P. Budhwar, and S. Gupta, ''AI-augmented HRM: Literature review and a proposed multilevel framework for future research,'' *Technological Forecasting Social Change*, vol. 193, Aug. 2023, Art. no. 122645.

[21] M. Bastida, A. V. García, M. Á. V. Taín, and M. Del Río Araujo, ''From automation to augmentation: Human resource's journey with artificial intelligence,'' *J. Ind. Inf. Integr.*, vol. 46, Jul. 2025, Art. no. 100872.

[22] A. Montgomery, I. Todorova, A. Baban, and E. Panagopoulou, ''Improving quality and safety in the hospital: The link between organizational culture, burnout, and quality of care,'' *Brit. J. Health Psychol.*, vol. 18, no. 3, pp. 656–662, Sep. 2013.

[23] A. Fenwick, G. Molnar, and P. Frangos, ''The critical role of HRM in AI-driven digital transformation: A paradigm shift to enable firms to move from AI implementation to human-centric adoption,'' *Discover Artif. Intell.*, vol. 4, no. 1, p. 34, May 2024.

[24] R. T. Sutton, D. Pincock, D. C. Baumgart, D. C. Sadowski, R. N. Fedorak, and K. I. Kroeker, ''An overview of clinical decision support systems: Benefits, risks, and strategies for success,'' *npj Digit. Med.*, vol. 3, no. 1, p. 17, Feb. 2020.

[25] M. A. Alabdali, S. A. Khan, M. Z. Yaqub, and M. A. Alshahrani, ''Harnessing the power of algorithmic human resource management and human resource strategic decision-making for achieving organizational success: An empirical analysis,'' *Sustainability*, vol. 16, no. 11, p. 4854, Jun. 2024.

[26] A. Haleem, M. Javaid, R. Pratap Singh, and R. Suman, ''Medical 4.0 technologies for healthcare: Features, capabilities, and applications,'' *Internet Things Cyber-Phys. Syst.*, vol. 2, pp. 12–30, Jun. 2022.

[27] A. Lamberti-Castronuovo, M. Valente, F. Barone-Adesi, I. Hubloue, and L. Ragazzoni, ''Primary health care disaster preparedness: A review of the literature and the proposal of a new framework,'' *Int. J. Disaster Risk Reduction*, vol. 81, Oct. 2022, Art. no. 103278.

[28] M. Eshghali, D. Kannan, N. Salmanzadeh-Meydani, and A. M. E. Sikaroudi, ''Machine learning based integrated scheduling and rescheduling for elective and emergency patients in the operating theatre,'' *Ann. Oper. Res.*, vol. 332, nos. 1–3, pp. 989–1012, Jan. 2024.

[29] R. R. Dangi, A. Sharma, and V. Vageriya, ''Transforming healthcare in low-resource settings with artificial intelligence: Recent developments and outcomes,'' *Public Health Nursing*, vol. 42, no. 2, pp. 1017–1030, Mar. 2025.

[30] G. Improta, V. Mazzella, D. Vecchione, S. Santini, and M. Triassi, ''Fuzzy logic-based clinical decision support system for the evaluation of renal function in post-transplant patients,'' *J. Eval. Clin. Pract.*, vol. 26, no. 4, pp. 1224–1234, Aug. 2020.

[31] M. Shoaib, B. Shah, T. Hussain, B. Yang, A. Ullah, J. Khan, and F. Ali, ''A deep learning-assisted visual attention mechanism for anomaly detection in videos,'' *Multimedia Tools Appl.*, vol. 83, no. 29, pp. 73363–73390, Dec. 2023.

[32] Y. Li, Y. Yao, J. Lin, and N. Wang, ''A deep learning algorithm based on CNN-LSTM framework for predicting cancer drug sales volume,'' 2025, *arXiv:2506.21927*.

[33] N. Karunanayake, ''Next-generation agentic AI for transforming healthcare,'' *Informat. Health*, vol. 2, no. 2, pp. 73–83, Sep. 2025.

[34] E. Y. Hidayat, Y. P. Astuti, I. N. Dewi, A. Salam, M. A. Soeleman, Z. A. Hasibuan, and A. S. Yousif, ''Genetic algorithm-based convolutional neural network feature engineering for optimizing coronary heart disease prediction performance,'' *Healthcare Informat. Res.*, vol. 30, no. 3, pp. 234–243, Jul. 2024.

[35] T. Iqbal, A. J. Simpkin, D. Roshan, N. Glynn, J. Killilea, J. Walsh, G. Molloy, S. Ganly, H. Ryman, E. Coen, A. Elahi, W. Wijns, and A. Shahzad, ''Stress monitoring using wearable sensors: A pilot study and stress-predict dataset,'' *Sensors*, vol. 22, no. 21, p. 8135, Oct. 2022.

[36] J.-A. Sim, X. Huang, M. R. Horan, J. N. Baker, and I.-C. Huang, ''Using natural language processing to analyze unstructured patient-reported outcomes data derived from electronic health records for cancer populations: A systematic review,'' *Expert Rev. Pharmacoeconomics Outcomes Res.*, vol. 24, no. 4, pp. 467–475, Apr. 2024.

[37] A. S. Albahri, Y. L. Khaleel, M. A. Habeeb, R. D. Ismael, Q. A. Hameed, M. Deveci, R. Z. Homod, O. S. Albahri, A. H. Alamoodi, and L. Alzubaidi, ''A systematic review of trustworthy artificial intelligence applications in natural disasters,'' *Comput. Electr. Eng.*, vol. 118, Sep. 2024, Art. no. 109409.

[38] R. Kumar, A. Singh, A. S. A. Kassar, M. I. Humaida, S. Joshi, and M. Sharma, ''Leveraging artificial intelligence to achieve sustainable public healthcare services in Saudi Arabia: A systematic literature review of critical success factors,'' *Comput. Model. Eng. Sci.*, vol. 142, no. 2, pp. 1289–1349, 2025.

[39] R. Liu, X. Xu, Y. Shen, A. Zhu, C. Yu, T. Chen, and Y. Zhang, ''Enhanced detection classification via clustering SVM for various robot collaboration task,'' in *Proc. 6th Int. Conf. Commun., Inf. Syst. Comput. Eng. (CISCE)*, May 2024, pp. 1121–1125.

[40] H. S. Ullah and S. Aslam, ''Blockchain in healthcare and medicine: A contemporary research of applications, challenges, and future perspectives,'' 2020, *arXiv:2004.06795*.

[41] I. T. Akbasli, A. Z. Birbilen, and O. Teksam, ''Artificial intelligence-driven forecasting and shift optimization for pediatric emergency department crowding,'' *JAMIA Open*, vol. 8, no. 2, pp. 125–138, Mar. 2025.

[42] G. Arji, L. Erfannia, S. Alirezaei, and M. Hemmat, ''A systematic literature review and analysis of deep learning algorithms in mental disorders,'' *Informat. Med. Unlocked*, vol. 40, Dec. 2023, Art. no. 101284.

[43] S. Manafi Varkiani, F. Pattarin, T. Fabbri, and G. Fantoni, ''Predicting employee attrition and explaining its determinants,'' *Expert Syst. Appl.*, vol. 272, May 2025, Art. no. 126575.

[44] S. G. Paul, A. Saha, M. Z. Hasan, S. R. H. Noori, and A. Moustafa, ''A systematic review of graph neural network in healthcare-based applications: Recent advances, trends, and future directions,'' *IEEE Access*, vol. 12, pp. 15145–15170, 2024.

[45] Y. Sun and Y. Gao, ''A multi-objective particle swarm optimization algorithm based on Gaussian mutation and an improved learning strategy,'' *Mathematics*, vol. 7, no. 2, p. 148, Feb. 2019.

[46] S. Maleki Varnosfaderani and M. Forouzanfar, ''The role of AI in hospitals and clinics: Transforming healthcare in the 21st century,'' *Bioengineering*, vol. 11, no. 4, p. 337, Mar. 2024.

[47] X. Wang, H. Yu, S. Kold, O. Rahbek, and S. Bai, ''Wearable sensors for activity monitoring and motion control: A review,'' *Biomimetic Intell. Robot.*, vol. 3, no. 1, Mar. 2023, Art. no. 100089.

[48] N. Savanović, A. Toskovic, A. Petrovic, M. Zivkovic, R. Damaševičius, L. Jovanovic, N. Bacanin, and B. Nikolic, ''Intrusion detection in healthcare 4.0 Internet of Things systems via metaheuristics optimized machine learning,'' *Sustainability*, vol. 15, no. 16, p. 12563, Aug. 2023.

[49] S. Golubovic, L. Jovanovic, B. Radomirovic, A. Njegus, M. Zivkovic, and N. Bacanin, ''Evolving deep neural network architectures by sine cosine algorithm for healthcare 4.0,'' in *Proc. IEEE Int. Conf. Contemp. Comput. Commun. (InC4)*, Apr. 2023, pp. 1–6.

[50] F. Markovic, L. Jovanovic, P. Spalevic, J. Kaljevic, M. Zivkovic, V. Simic, H. Shaker, and N. Bacanin, ''Parkinsons detection from gait time series classification using modified metaheuristic optimized long short term memory,'' *Neural Process. Lett.*, vol. 57, no. 1, p. 14, Feb. 2025.

[51] L. Jovanovic, N. Bacanin, M. Zivkovic, M. Antonijevic, A. Petrovic, and T. Zivkovic, ''Anomaly detection in ECG using recurrent networks optimized by modified metaheuristic algorithm,'' in *Proc. 31st Telecommun. Forum (TELFOR)*, Nov. 2023, pp. 1–4.

[52] L. Jovanovic, S. Golubovic, N. Bacanin, G. Kunjadic, M. Antonijevic, and M. Zivkovic, ''Performance evaluation of metaheuristics-tuned deep neural networks for healthcare 4.0,'' in *Proc. Int. Conf. Comput. Sci. Sustain. Technol.*, 2023, pp. 1–14.

[53] S. J. Villoth, P. Dabic, T. Zivkovic, M. Zivkovic, S. Andjelic, M. Mravik, V. Simic, M. Abdel-Salam, and N. Bacanin, ''Cardiovascular disease risk prediction utilizing two-tier classification framework optimized with adapted variable neighborhood search algorithm,'' *Algorithms*, vol. 19, no. 2, p. 130, Feb. 2026.

[54] L. Anicin, S. Andjelic, M. M. Blagojevic, D. Bulaja, M. Zivkovic, T. Zivkovic, M. Antonijevic, and N. Bacanin, ''Metaheuristic-driven dual-layer model for classifying Alzheimer's disease stages,'' *Frontiers Comput. Neurosci.*, vol. 20, pp. 20–2026, Feb. 2026.

[55] P. Dabic, J. Petrovic, M. Zivkovic, T. Zivkovic, V. Simic, D. Pamucar, M. Dobrojevic, and N. Bacanin, ''Hospital admission classification of cardiac patients utilizing metaheuristics-optimized two tier framework,'' *Int. J. Comput. Intell. Syst.*, vol. 19, no. 1, pp. 17–32, Dec. 2025.

[56] N. R. Hander, R. Erschens, T. Klein, M. N. Jarczok, N. Mulfinger, M. A. Rieger, F. Junne, P. Angerer, B. Puschner, A. Müller, J. Küllenberg, I. Maatouk, S. Süß, E. Gesang, S. Ruhle, H. Gündel, and E. Rothermund-Nassir, ''Working conditions, job stress and work-related consequences among hospital employees—Differences by professional group, working hours and job levels: A cross-sectional study,'' *PLoS ONE*, vol. 21, no. 3, Mar. 2026, Art. no. e0343567.

[57] J. A. Zafra-Agea, E. Ramírez-Baraldes, E. Maldonado-Manzano, N. Obradors-Rial, A. Puiggrós-Binefa, and E. Colillas-Malet, ''Working conditions and well-being of school nurses in spain: Impact on job satisfaction and professional quality of life,'' *Healthcare*, vol. 13, no. 3, p. 323, Feb. 2025.

[58] M. Sánchez-Sellero, ''Synthe tic indic ators of qualit y of subjec tive life in the EU?: RUR Al and urban are as,'' *Prague Econ. Pap.*, vol. 30, no. 5, pp. 529–551, 2021.

[59] Z. Zhang and M. Tao, ''Deep learning for wireless coded caching with unknown and time-variant content popularity,'' *IEEE Trans. Wireless Commun.*, vol. 20, no. 2, pp. 1152–1163, Feb. 2021.

MUHAMMAD SHOAIB received the master's degree in computer science from the Department of Computer Science, Islamia College Peshawar, Khyber Pakhtunkhwa, Pakistan. He is currently pursuing the Ph.D. degree in computer science with Sarhad University of Science and Technology Peshawar, Khyber Pakhtunkhwa. He is a Lecturer with the Department of Computer Science, CECOS University of IT and Emerging Sciences, Peshawar, Khyber Pakhtunkhwa, where he is actively engaged in teaching and research activities. He is also an Accomplished Researcher with expertise in medical image and signal processing, machine learning, computer vision, image understanding, pattern recognition, video scene understanding, and human activity analysis. With his extensive knowledge and expertise, he has contributed significantly to the field of computer science, having published several research articles in reputed academic journals. His work has been well-received by the academic community and has helped to advance our understanding of these complex topics.

MUHAMMAD ISMAIL MOHMAND is currently an Associate Professor with the Department of Computer Engineering, Faculty of Engineering and Natural Sciences, Istanbul Atlas University, Türkiye. He has international teaching experience in countries, such as China, Malaysia, U.K., Kazakhstan, and Pakistan. He has published original research articles in various international and reputed journals. He is actively engaged in both teaching and research. His research interests include computer networking, artificial intelligence, cybersecurity, the Internet of Things (IoT), and software engineering.

NASIR SAYED received the master's degree in computer science from the Department of Computer Science, Islamia College Peshawar, Khyber Pakhtunkhwa, Pakistan, where he is currently pursuing the Ph.D. degree in computer science. His research interests focus on intelligent computer vision-based systems, including human activity recognition, video analysis, deepfake detection, and medical image analysis. His current research involves the application of deep learning and machine learning techniques for feature representation, pat-tern recognition, and robust decision-support systems in real-world and healthcare-oriented environments.

INAYAT KHAN received the Ph.D. degree in computer science from the Department of Computer Science, University of Peshawar, Pakistan. He is currently an Assistant Professor with the Department of Computer Science, University of Engineering and Technology Mardan, Pakistan. His current research is based on the design and development of context-aware adaptive user interfaces for minimizing drivers' distractions. His other research interests include lifelogging, healthcare, deep learning, ubiquitous computing, accessibility, and mobile-based assistive systems for people with special needs. He has published several articles in international journals and conferences in these areas.

SALMAN JAN received the master's degree in computer science from the University of Peshawar and the Ph.D. degree from MIIT, Universiti Kuala Lumpur, in 2019. He is currently an Assistant Professor with the Faculty of Computer Studies, Arab Open University, Bahrain. His Ph.D. thesis was on Android malware analysis and its integration with blockchain for preserving integrity of the behavioral logs and ultimately to secure the classification results of the behavioral logs through the employed deep learning models, including DCGAN, CNN, and FCNN. He has acquired skills and published in several well-reputed journals in areas of machine learning, deep learning, artificial intelligence, blockchain, the IoT, and security augmented and virtual reality. He is also actively working on international projects in the capacity of consultant and has successfully completed several international projects.

IT EE LEE received the Bachelor of Engineering degree (Hons.) in electronics and the M.Eng.Sc. degree from Multimedia University, Cyberjaya, Malaysia, in 2003 and 2009, respectively, and the Ph.D. degree in optical wireless communi-cation from Northumbria University, Newcastle upon Tyne, U.K., in 2014. She is currently an Assistant Professor with the Faculty of Artificial Intelligence and Engineering, Multimedia University. She has authored several publications in international peer-reviewed journals, international conferences, and book chapter. Her research interests include wireless optical communications, the Internet of Things, machine learning, and sustainable technologies. She contributes as a technical program committee member for international conferences and a technical reviewer for various distinguished international peer-reviewed journals.